

Variational Inference — Basics

From Bayes' rule to score matching and diffusion models

Andrija Petrović

August 1, 2026

Outline

The core problem: Bayesian inference

A brief tour of MCMC

Foundations of variational inference

Mean-field variational inference

Black box variational inference

Going deeper: hierarchical VAEs

Expressive approximations: normalizing flows

Score matching and diffusion models

Conclusion

Where we are

The core problem: Bayesian inference

A brief tour of MCMC

Foundations of variational inference

Mean-field variational inference

Black box variational inference

Going deeper: hierarchical VAEs

Expressive approximations: normalizing flows

Score matching and diffusion models

Conclusion

The goal: posterior inference

Bayes' rule

The cornerstone of Bayesian machine learning: how to update our beliefs about parameters (or latent variables) z after observing data x :

$$\underbrace{p(z | x)}_{\text{posterior}} = \frac{\overbrace{p(x | z)}^{\text{likelihood}} \overbrace{p(z)}^{\text{prior}}}{\underbrace{p(x)}_{\text{evidence}}}$$

- $p(z | x)$ — what we believe about z *after* seeing data. This is what we want.

The goal: posterior inference

Bayes' rule

The cornerstone of Bayesian machine learning: how to update our beliefs about parameters (or latent variables) z after observing data x :

$$\underbrace{p(z | x)}_{\text{posterior}} = \frac{\overbrace{p(x | z)}^{\text{likelihood}} \overbrace{p(z)}^{\text{prior}}}{\underbrace{p(x)}_{\text{evidence}}}$$

- $p(z | x)$ — what we believe about z *after* seeing data. This is what we want.
- $p(x | z)$ — our model of how the data was generated.

The goal: posterior inference

Bayes' rule

The cornerstone of Bayesian machine learning: how to update our beliefs about parameters (or latent variables) z after observing data x :

$$\underbrace{p(z | x)}_{\text{posterior}} = \frac{\overbrace{p(x | z)}^{\text{likelihood}} \overbrace{p(z)}^{\text{prior}}}{\underbrace{p(x)}_{\text{evidence}}}$$

- $p(z | x)$ — what we believe about z *after* seeing data. This is what we want.
- $p(x | z)$ — our model of how the data was generated.
- $p(z)$ — what we believed *before* seeing any data.

The goal: posterior inference

Bayes' rule

The cornerstone of Bayesian machine learning: how to update our beliefs about parameters (or latent variables) z after observing data x :

$$\underbrace{p(z | x)}_{\text{posterior}} = \frac{\overbrace{p(x | z)}^{\text{likelihood}} \overbrace{p(z)}^{\text{prior}}}{\underbrace{p(x)}_{\text{evidence}}}$$

- $p(z | x)$ — what we believe about z *after* seeing data. This is what we want.
- $p(x | z)$ — our model of how the data was generated.
- $p(z)$ — what we believed *before* seeing any data.
- $p(x)$ — the probability of the data, averaged over all possible z .

Where do these posteriors come from?

The same object shows up everywhere once you look for it.

Bayesian neural net

z = network weights

$p(z | x)$ tells us *how uncertain* the network is — essential for medicine, finance, control.

Latent variable models

z = latent code of an image

The encoder of a VAE *is* an approximate posterior $q(z | x)$.

Scientific models

z = physical parameters

Epidemic rates, cosmological constants, drug effects — we need the whole posterior, not one number.

In every case the numerator is easy and the denominator is the problem.

The intractability of the evidence

The evidence integral

$$p(x) = \int p(x, z) dz = \int p(x | z) p(z) dz$$

- A high-dimensional integral: for a neural network z has millions of dimensions. Numerical quadrature is hopeless (cost grows exponentially in $\dim z$).

The intractability of the evidence

The evidence integral

$$p(x) = \int p(x, z) dz = \int p(x | z) p(z) dz$$

- A high-dimensional integral: for a neural network z has millions of dimensions. Numerical quadrature is hopeless (cost grows exponentially in $\dim z$).
- Closed form only in special (conjugate) cases:

Prior	Likelihood	Posterior
Beta	Bernoulli / Binomial	Beta
Normal	Normal (known variance)	Normal
Gamma	Poisson	Gamma
Dirichlet	Categorical	Dirichlet

The intractability of the evidence

The evidence integral

$$p(x) = \int p(x, z) dz = \int p(x | z) p(z) dz$$

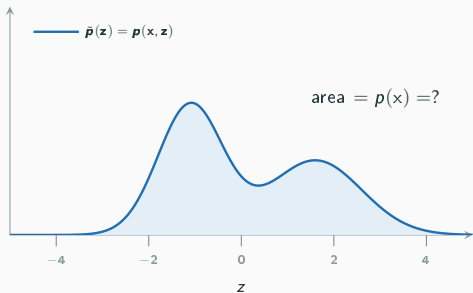
- A high-dimensional integral: for a neural network z has millions of dimensions. Numerical quadrature is hopeless (cost grows exponentially in $\dim z$).
- Closed form only in special (conjugate) cases:

Prior	Likelihood	Posterior
Beta	Bernoulli / Binomial	Beta
Normal	Normal (known variance)	Normal
Gamma	Poisson	Gamma
Dirichlet	Categorical	Dirichlet

- Without $p(x)$ we only know the posterior *up to a constant*:

$$p(z | x) \propto p(x, z)$$

What “known up to a constant” looks like



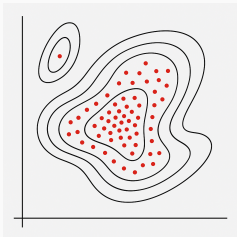
- We can **evaluate** $\tilde{p}(z) = p(x, z)$ at any point, cheaply.
- We cannot **normalise** it: the shaded area is the intractable integral.
- Everything that follows is a way of living without that number.

Two escape routes

Draw samples that behave as if they came from p (MCMC),
or fit a normalised q that looks like p (VI).

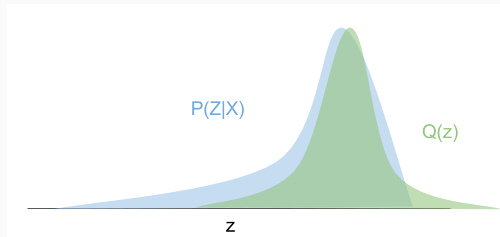
Two paths forward

Stochastic approximation



- Generate samples that approximate the posterior.
- **Key method:** Markov chain Monte Carlo (MCMC).
- Asymptotically exact, but slow.

Deterministic approximation



- Pose inference as an *optimisation* problem.
- **Key method:** variational inference (VI).
- Find the closest distribution to the posterior inside a simple family.

Where we are

The core problem: Bayesian inference

A brief tour of MCMC

Foundations of variational inference

Mean-field variational inference

Black box variational inference

Going deeper: hierarchical VAEs

Expressive approximations: normalizing flows

Score matching and diffusion models

Conclusion

The core idea

Build a Markov chain whose states are values of z and whose *stationary distribution* is exactly the posterior $p(z | x)$. Run it long enough and its states are samples from the posterior.

- We never need $p(x)$: the algorithms only ever look at *ratios* of the unnormalised density $p(x, z)$.

Markov chain Monte Carlo

The core idea

Build a Markov chain whose states are values of z and whose *stationary distribution* is exactly the posterior $p(z | x)$. Run it long enough and its states are samples from the posterior.

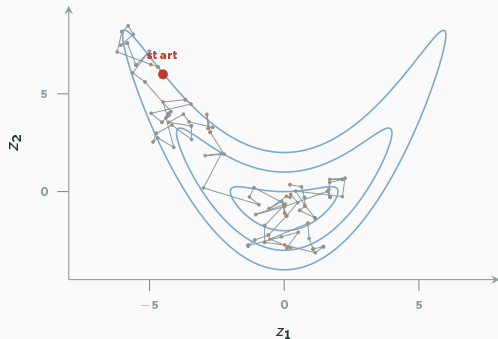
- We never need $p(x)$: the algorithms only ever look at *ratios* of the unnormalised density $p(x, z)$.
- **Metropolis–Hastings, in three lines:**
 1. Start at some $z^{(0)}$.
 2. Propose $z' \sim g(z' | z^{(t)})$.
 3. Accept with probability

$$\alpha = \min \left(1, \frac{p(x, z') g(z^{(t)} | z')}{p(x, z^{(t)}) g(z' | z^{(t)})} \right)$$

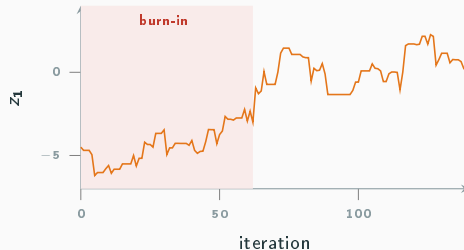
otherwise stay where you are (and record the current state again).

MCMC in action: a random walk that learns the shape

Chain path on a banana-shaped posterior



Trace of z_1



- The first ~ 60 steps are *burn-in*: the chain is still travelling towards the bulk of the posterior. They must be discarded.
- Consecutive samples are correlated — flat stretches are rejected proposals.

Derivation: the goal is stationarity

What property must our chain have?

We want the distribution of $z^{(t)}$ to stop changing and to equal the target $\pi(z) \equiv p(z | x)$.
That is the **stationary distribution**.

Let $T(z' | z)$ be the transition kernel: the probability of moving from z to z' .

Derivation: the goal is stationarity

What property must our chain have?

We want the distribution of $z^{(t)}$ to stop changing and to equal the target $\pi(z) \equiv p(z | x)$.
That is the **stationary distribution**.

Let $T(z' | z)$ be the transition kernel: the probability of moving from z to z' .

The stationarity condition

π is stationary for T if

$$\pi(z') = \int \pi(z) T(z' | z) dz$$

In words: if the states are already distributed according to π , one more step leaves that distribution unchanged.

Derivation: the goal is stationarity

What property must our chain have?

We want the distribution of $z^{(t)}$ to stop changing and to equal the target $\pi(z) \equiv p(z | x)$.
That is the **stationary distribution**.

Let $T(z' | z)$ be the transition kernel: the probability of moving from z to z' .

The stationarity condition

π is stationary for T if

$$\pi(z') = \int \pi(z) T(z' | z) dz$$

In words: if the states are already distributed according to π , one more step leaves that distribution unchanged.

Enforcing this directly is hard — it is an integral equation. We need a sufficient condition that is easy to check.

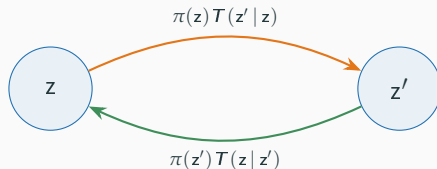
Derivation: a sufficient condition — detailed balance

The detailed balance equation

T satisfies detailed balance with respect to π if

$$\pi(z) T(z' | z) = \pi(z') T(z | z')$$

In words: in the stationary state, the flow of probability from z to z' exactly balances the flow back.



Derivation: detailed balance $\Rightarrow$ stationarity

Proof

Integrate the detailed balance equation over z :

$$\begin{aligned}\int \pi(z) T(z' | z) dz &= \int \pi(z') T(z | z') dz \\ &= \pi(z') \int T(z | z') dz && (\pi(z') \text{ does not depend on } z) \\ &= \pi(z') \cdot 1 \\ &= \pi(z') \blacksquare\end{aligned}$$

The left-hand side is exactly the stationarity condition.

Our job is now concrete: **construct a kernel T that satisfies detailed balance.**

Derivation: constructing Metropolis–Hastings

A transition has two parts: propose, then accept.

- **Proposal:** $g(z' | z)$ — anything we can sample from.
- **Acceptance:** accept with probability $A(z', z)$.

So for $z' \neq z$: $T(z' | z) = g(z' | z)A(z', z)$.

Derivation: constructing Metropolis–Hastings

A transition has two parts: propose, then accept.

- **Proposal:** $g(z' | z)$ — anything we can sample from.
- **Acceptance:** accept with probability $A(z', z)$.

So for $z' \neq z$: $T(z' | z) = g(z' | z)A(z', z)$.

Plug into detailed balance:

$$\pi(z)g(z' | z)A(z', z) = \pi(z')g(z | z')A(z, z')$$

Derivation: constructing Metropolis–Hastings

A transition has two parts: propose, then accept.

- **Proposal:** $g(z' | z)$ — anything we can sample from.
- **Acceptance:** accept with probability $A(z', z)$.

So for $z' \neq z$: $T(z' | z) = g(z' | z)A(z', z)$.

Plug into detailed balance:

$$\pi(z)g(z' | z)A(z', z) = \pi(z')g(z | z')A(z, z')$$

which constrains only the *ratio* of the acceptance probabilities:

$$\frac{A(z', z)}{A(z, z')} = \frac{\pi(z')g(z | z')}{\pi(z)g(z' | z)}$$

Derivation: constructing Metropolis–Hastings

A transition has two parts: propose, then accept.

- **Proposal:** $g(z' | z)$ — anything we can sample from.
- **Acceptance:** accept with probability $A(z', z)$.

So for $z' \neq z$: $T(z' | z) = g(z' | z)A(z', z)$.

Plug into detailed balance:

$$\pi(z)g(z' | z)A(z', z) = \pi(z')g(z | z')A(z, z')$$

which constrains only the *ratio* of the acceptance probabilities:

$$\frac{A(z', z)}{A(z, z')} = \frac{\pi(z')g(z | z')}{\pi(z)g(z' | z)}$$

The **Metropolis choice** satisfies it while accepting as often as possible:

$$A(z', z) = \min \left(1, \frac{\pi(z')g(z | z')}{\pi(z)g(z' | z)} \right)$$

Derivation: the normaliser cancels

Why this is practical

Since $\pi(z) = p(z|x) = \frac{p(x, z)}{p(x)}$, the unknown constant $p(x)$ appears in both numerator and denominator and cancels:

$$\alpha = \min \left(1, \frac{p(x, z') g(z|z')}{p(x, z) g(z'|z)} \right)$$

This is exactly the Metropolis–Hastings acceptance rule from three slides ago.

We can sample from a distribution we cannot even evaluate.

Metropolis–Hastings, in code

```
import numpy as np

def metropolis_hastings(log_joint, z0, n_steps, step=0.5):
    """log_joint(z) returns log p(x, z) -- UNNORMALISED is fine."""
    z, lp = z0, log_joint(z0)
    chain, n_accept = [z], 0

    for _ in range(n_steps):
        z_new = z + step * np.random.randn(*z.shape)    # symmetric proposal
        lp_new = log_joint(z_new)

        # log alpha = log p(x, z') - log p(x, z); g cancels because g is symmetric
        if np.log(np.random.rand()) < lp_new - lp:
            z, lp = z_new, lp_new                        # accept
            n_accept += 1

        chain.append(z)                                  # note: append even on reject

    print("acceptance rate:", n_accept / n_steps)        # aim for ~0.25 - 0.5
    return np.array(chain)
```

The whole algorithm is six lines. The hard part is choosing step and knowing when to stop.

MCMC: pros and cons

Pros

- **Asymptotically exact:** given enough time, samples come from the true posterior.
- **General:** works for almost any model you can evaluate.
- **Faithful:** captures multimodality, correlations, heavy tails.

Cons

- **Slow:** long burn-in, and consecutive samples are correlated.
- **Hard to scale:** every step touches the whole dataset; awkward to parallelise.
- **No reliable stopping rule:** convergence diagnostics ($\hat{R}$, ESS) can only ever falsify convergence.

MCMC's slowness is what motivates variational inference.

Where we are

The core problem: Bayesian inference

A brief tour of MCMC

Foundations of variational inference

Mean-field variational inference

Black box variational inference

Going deeper: hierarchical VAEs

Expressive approximations: normalizing flows

Score matching and diffusion models

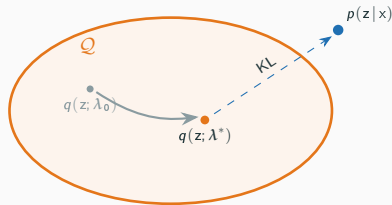
Conclusion

The core idea of variational inference

From integration to optimisation

VI replaces the intractable integral by an optimisation problem — and we are very good at optimisation.

1. **Choose a family** of simple distributions
 $q(z; \lambda)$ — the *variational family* $\mathcal{Q}$.
e.g. all Gaussians: $\lambda = \{\mu, \Sigma\}$.



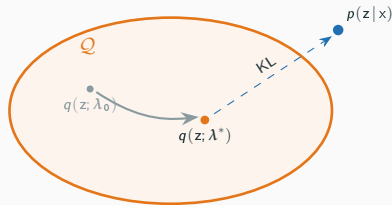
Optimisation walks λ through $\mathcal{Q}$ to the point nearest p .

The core idea of variational inference

From integration to optimisation

VI replaces the intractable integral by an optimisation problem — and we are very good at optimisation.

1. **Choose a family** of simple distributions
 $q(z; \lambda)$ — the *variational family* $\mathcal{Q}$.
e.g. all Gaussians: $\lambda = \{\mu, \Sigma\}$.
2. **Find the member closest** to the true posterior: λ^* minimises a divergence to $p(z | x)$.



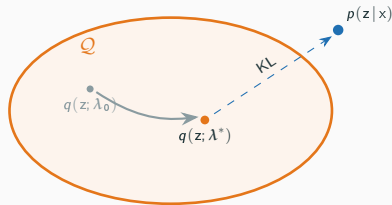
Optimisation walks λ through $\mathcal{Q}$ to the point nearest p .

The core idea of variational inference

From integration to optimisation

VI replaces the intractable integral by an optimisation problem — and we are very good at optimisation.

1. **Choose a family** of simple distributions $q(z; \lambda)$ — the *variational family* $\mathcal{Q}$.
e.g. all Gaussians: $\lambda = \{\mu, \Sigma\}$.
2. **Find the member closest** to the true posterior: λ^* minimises a divergence to $p(z | x)$.
3. **Use** $q(z; \lambda^*)$ everywhere you would have used the posterior.



Optimisation walks λ through $\mathcal{Q}$ to the point nearest p .

Measuring closeness: the KL divergence

Kullback–Leibler divergence

$$\text{KL}(q \parallel p) = \int q(z) \log \frac{q(z)}{p(z|x)} dz = \mathbb{E}_{q(z)} \left[\log \frac{q(z)}{p(z|x)} \right]$$

- $\text{KL}(q \parallel p) \geq 0$, with equality *iff* $q = p$ (Jensen's inequality).
- It is **not** symmetric: $\text{KL}(q \parallel p) \neq \text{KL}(p \parallel q)$. This asymmetry is not a technicality — it changes the answer.
- It is not a metric (no triangle inequality).

Measuring closeness: the KL divergence

Kullback–Leibler divergence

$$\text{KL}(q \parallel p) = \int q(z) \log \frac{q(z)}{p(z \mid x)} dz = \mathbb{E}_{q(z)} \left[\log \frac{q(z)}{p(z \mid x)} \right]$$

- $\text{KL}(q \parallel p) \geq 0$, with equality *iff* $q = p$ (Jensen's inequality).
- It is **not** symmetric: $\text{KL}(q \parallel p) \neq \text{KL}(p \parallel q)$. This asymmetry is not a technicality — it changes the answer.
- It is not a metric (no triangle inequality).

The goal:

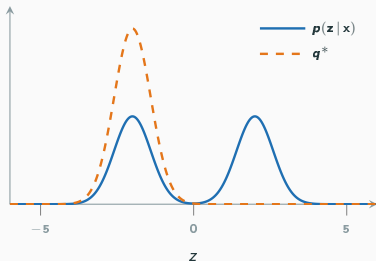
$$\lambda^* = \arg \min_{\lambda} \text{KL} (q(z; \lambda) \parallel p(z \mid x))$$

Note the order: q first. VI minimises the *reverse* KL. Why that matters is next.

Which KL? Mode-seeking vs. mass-covering

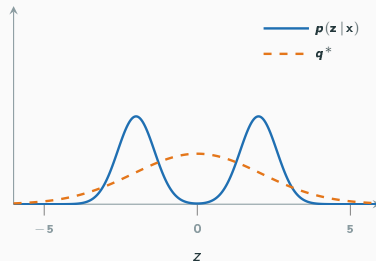
Fit a *single* Gaussian q to a bimodal p . The two KL directions disagree.

Reverse KL: $KL(q \parallel p)$ — what VI does



Zero-forcing. q must be small wherever p is small (else $\log \frac{q}{p} \rightarrow \infty$). It picks *one* mode and **underestimates variance**.

Forward KL: $KL(p \parallel q)$ — what EP / MLE do



Mass-covering. q must be large wherever p is (moment matching). It spans both modes and puts mass where there is none.

VI uses reverse KL because it is the one we can actually compute.

The problem with minimising KL directly

Expand the definition:

$$\begin{aligned}\text{KL}(q \parallel p) &= \mathbb{E}_q [\log q(z; \lambda) - \log p(z \mid x)] \\ &= \mathbb{E}_q [\log q(z; \lambda)] - \mathbb{E}_q \left[\log \frac{p(x, z)}{p(x)} \right] \\ &= \mathbb{E}_q [\log q(z; \lambda)] - \mathbb{E}_q [\log p(x, z)] + \underbrace{\log p(x)}_{\text{constant in } \lambda}\end{aligned}$$

The problem with minimising KL directly

Expand the definition:

$$\begin{aligned}\text{KL}(q \parallel p) &= \mathbb{E}_q [\log q(z; \lambda) - \log p(z \mid x)] \\ &= \mathbb{E}_q [\log q(z; \lambda)] - \mathbb{E}_q \left[\log \frac{p(x, z)}{p(x)} \right] \\ &= \mathbb{E}_q [\log q(z; \lambda)] - \mathbb{E}_q [\log p(x, z)] + \underbrace{\log p(x)}_{\text{constant in } \lambda}\end{aligned}$$

The catch

To evaluate the objective we need $\log p(x)$ — the very quantity that was intractable in the first place.

The problem with minimising KL directly

Expand the definition:

$$\begin{aligned}\text{KL}(q \parallel p) &= \mathbb{E}_q [\log q(z; \lambda) - \log p(z \mid x)] \\ &= \mathbb{E}_q [\log q(z; \lambda)] - \mathbb{E}_q \left[\log \frac{p(x, z)}{p(x)} \right] \\ &= \mathbb{E}_q [\log q(z; \lambda)] - \mathbb{E}_q [\log p(x, z)] + \underbrace{\log p(x)}_{\text{constant in } \lambda}\end{aligned}$$

The catch

To evaluate the objective we need $\log p(x)$ — the very quantity that was intractable in the first place.

...but notice: it is a *constant* with respect to λ . That single observation saves us.

The evidence lower bound (ELBO)

Rearranging the previous line:

$$\log p(x) = \underbrace{\mathbb{E}_q[\log p(x, z) - \log q(z; \lambda)]}_{\mathcal{L}(\lambda) = \text{ELBO}} + \text{KL} (q(z; \lambda) \parallel p(z | x))$$

The evidence lower bound (ELBO)

Rearranging the previous line:

$$\log p(\mathbf{x}) = \underbrace{\mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z}) - \log q(\mathbf{z}; \boldsymbol{\lambda})]}_{\mathcal{L}(\boldsymbol{\lambda}) = \text{ELBO}} + \text{KL} (q(\mathbf{z}; \boldsymbol{\lambda}) \parallel p(\mathbf{z} | \mathbf{x}))$$

Since $\text{KL} \geq 0$:

$$\log p(\mathbf{x}) \geq \mathcal{L}(\boldsymbol{\lambda})$$

The evidence lower bound (ELBO)

Rearranging the previous line:

$$\log p(x) = \underbrace{\mathbb{E}_q[\log p(x, z) - \log q(z; \lambda)]}_{\mathcal{L}(\lambda) = \text{ELBO}} + \text{KL}(q(z; \lambda) \parallel p(z | x))$$

Since $\text{KL} \geq 0$:

$$\log p(x) \geq \mathcal{L}(\lambda)$$

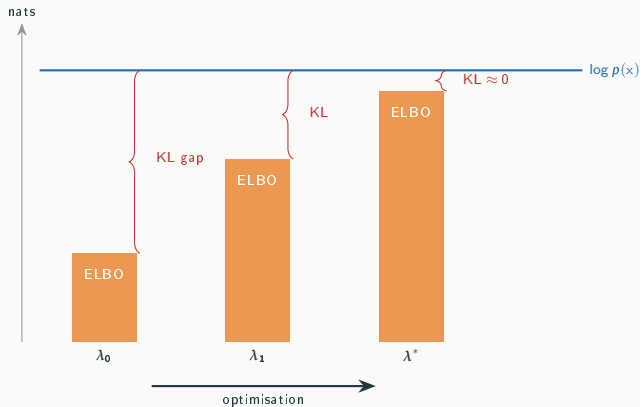
The key insight

$\log p(x)$ is *fixed*. So pushing the ELBO up is *exactly* the same as pushing the KL down:

$$\arg \max_{\lambda} \mathcal{L}(\lambda) = \arg \min_{\lambda} \text{KL}(q \parallel p)$$

and $\mathcal{L}$ contains only things we can compute: the joint and our own q .

The ELBO, geometrically



The two pieces always sum to the same height, so optimisation trades KL for ELBO. The residual gap at λ^* is the price of the family $\mathcal{Q}$.

Deconstructing the ELBO (1): energy and entropy

$$\mathcal{L}(\lambda) = \underbrace{\mathbb{E}_q[\log p(x, z)]}_{\text{energy}} - \underbrace{\mathbb{E}_q[\log q(z; \lambda)]}_{\text{entropy } H(q)}$$

- The **energy** term pulls q towards configurations of z that explain the data well. On its own it would collapse q to a spike at the MAP.

Deconstructing the ELBO (1): energy and entropy

$$\mathcal{L}(\lambda) = \underbrace{\mathbb{E}_q[\log p(x, z)]}_{\text{energy}} - \underbrace{\mathbb{E}_q[\log q(z; \lambda)]}_{\text{entropy } H(q)}$$

- The **energy** term pulls q towards configurations of z that explain the data well. On its own it would collapse q to a spike at the MAP.
- The **entropy** term pushes q to spread out. On its own it would flatten q completely.

Deconstructing the ELBO (1): energy and entropy

$$\mathcal{L}(\lambda) = \underbrace{\mathbb{E}_q[\log p(x, z)]}_{\text{energy}} - \underbrace{\mathbb{E}_q[\log q(z; \lambda)]}_{\text{entropy } H(q)}$$

- The **energy** term pulls q towards configurations of z that explain the data well. On its own it would collapse q to a spike at the MAP.
- The **entropy** term pushes q to spread out. On its own it would flatten q completely.
- The ELBO is the tug-of-war between fitting the data and staying honest about uncertainty.

Deconstructing the ELBO (1): energy and entropy

$$\mathcal{L}(\lambda) = \underbrace{\mathbb{E}_q[\log p(x, z)]}_{\text{energy}} - \underbrace{\mathbb{E}_q[\log q(z; \lambda)]}_{\text{entropy } H(q)}$$

- The **energy** term pulls q towards configurations of z that explain the data well. On its own it would collapse q to a spike at the MAP.
- The **entropy** term pushes q to spread out. On its own it would flatten q completely.
- The ELBO is the tug-of-war between fitting the data and staying honest about uncertainty.

Drop the entropy term and Bayesian inference degenerates into MAP estimation.

Deconstructing the ELBO (2): reconstruction and regularisation

$$\mathcal{L}(\lambda) = \underbrace{\mathbb{E}_q[\log p(x | z)]}_{\text{reconstruction}} - \underbrace{\text{KL} (q(z; \lambda) \parallel p(z))}_{\text{regulariser}}$$

- **Reconstruction:** how well we recover x from latent codes z drawn from q .
- **Regulariser:** keeps the approximate posterior close to the prior, so the latent space stays usable.

This is *literally* the VAE loss function. The two forms are algebraically identical — the second just splits the joint as $p(x, z) = p(x | z)p(z)$.

1. **Define a model:** prior $p(z)$ and likelihood $p(x | z)$, giving the joint $p(x, z)$.

The VI recipe

1. **Define a model:** prior $p(z)$ and likelihood $p(x | z)$, giving the joint $p(x, z)$.
2. **Choose a variational family:** $q(z; \lambda)$ — tractable, but as expressive as you can afford.

The VI recipe

1. **Define a model:** prior $p(z)$ and likelihood $p(x | z)$, giving the joint $p(x, z)$.
2. **Choose a variational family:** $q(z; \lambda)$ — tractable, but as expressive as you can afford.
3. **Write the ELBO:** $\mathcal{L}(\lambda) = \mathbb{E}_q[\log p(x, z) - \log q(z; \lambda)]$.

The VI recipe

1. **Define a model:** prior $p(z)$ and likelihood $p(x | z)$, giving the joint $p(x, z)$.
2. **Choose a variational family:** $q(z; \lambda)$ — tractable, but as expressive as you can afford.
3. **Write the ELBO:** $\mathcal{L}(\lambda) = \mathbb{E}_q[\log p(x, z) - \log q(z; \lambda)]$.
4. **Optimise:** $\lambda^* = \arg \max_{\lambda} \mathcal{L}(\lambda)$ — by coordinate ascent (§CAVI) or stochastic gradients (§BBVI).

The VI recipe

1. **Define a model:** prior $p(z)$ and likelihood $p(x | z)$, giving the joint $p(x, z)$.
2. **Choose a variational family:** $q(z; \lambda)$ — tractable, but as expressive as you can afford.
3. **Write the ELBO:** $\mathcal{L}(\lambda) = \mathbb{E}_q[\log p(x, z) - \log q(z; \lambda)]$.
4. **Optimise:** $\lambda^* = \arg \max_{\lambda} \mathcal{L}(\lambda)$ — by coordinate ascent (§CAVI) or stochastic gradients (§BBVI).
5. **Use the result:** $q(z; \lambda^*)$ for prediction, uncertainty, model comparison.

The VI recipe

1. **Define a model:** prior $p(z)$ and likelihood $p(x | z)$, giving the joint $p(x, z)$.
2. **Choose a variational family:** $q(z; \lambda)$ — tractable, but as expressive as you can afford.
3. **Write the ELBO:** $\mathcal{L}(\lambda) = \mathbb{E}_q[\log p(x, z) - \log q(z; \lambda)]$.
4. **Optimise:** $\lambda^* = \arg \max_{\lambda} \mathcal{L}(\lambda)$ — by coordinate ascent (§CAVI) or stochastic gradients (§BBVI).
5. **Use the result:** $q(z; \lambda^*)$ for prediction, uncertainty, model comparison.

Steps 1–3 are modelling. Only step 4 is where the algorithms differ.

Where we are

The core problem: Bayesian inference

A brief tour of MCMC

Foundations of variational inference

Mean-field variational inference

Black box variational inference

Going deeper: hierarchical VAEs

Expressive approximations: normalizing flows

Score matching and diffusion models

Conclusion

Choosing the family: the mean-field assumption

The choice of Q trades approximation quality against optimisation difficulty.

Mean-field family

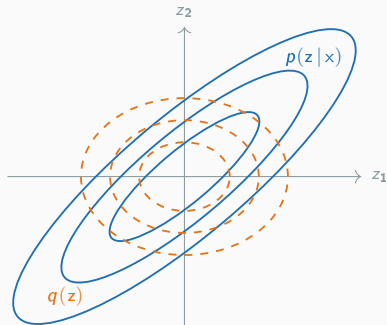
Assume the latent variables are *mutually independent* under q :

$$q(\mathbf{z}; \boldsymbol{\lambda}) = \prod_{j=1}^M q_j(z_j; \lambda_j)$$

Each factor q_j is free — we do *not* fix its parametric form in advance.

- Borrowed from statistical physics: each particle feels the *average* (mean) field of the others rather than each of them individually.
- The posterior's correlations are thrown away by construction.

Mean-field: what is lost



- The true posterior is stretched along a diagonal: z_1 and z_2 are strongly correlated.
- A factorised q can only be **axis-aligned**.
- The best axis-aligned fit is squeezed *inside* the true posterior, so

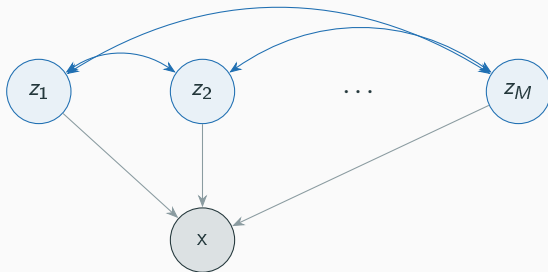
$$\text{Var}_q[z_j] < \text{Var}_p[z_j]$$

Consequence

Mean-field VI reports **overconfident** uncertainty. Its means are usually good; its error bars are usually too small.

Mean-field as a graphical model

True posterior $p(z|x)$: everything is coupled



Mean-field $q(z)$: all edges cut



Coordinate ascent VI (CAVI)

Optimise one factor at a time, holding the others fixed — like Gibbs sampling, but deterministic.

The CAVI update

$$q_j^*(z_j) \propto \exp \left(\mathbb{E}_{i \neq j} [\log p(\mathbf{x}, \mathbf{z})] \right)$$

where the expectation is over all *other* factors q_i , $i \neq j$.

- Each update maximises the ELBO in one coordinate block, so **the ELBO increases monotonically** — a free convergence check.
- It converges to a *local* optimum; the ELBO is generally non-convex.
- In conjugate exponential-family models the expectation is available in closed form and updates are a few lines of arithmetic.

Derivation of the CAVI update

Isolate the terms involving q_j (everything else is a constant):

$$\begin{aligned}\mathcal{L}(q_j) &= \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z})] - \mathbb{E}_q[\log q(\mathbf{z})] \\ &= \int q_j(z_j) \left(\int \prod_{i \neq j} q_i(z_i) \log p(\mathbf{x}, \mathbf{z}) dz_{-j} \right) dz_j - \int q_j \log q_j dz_j + \text{const} \\ &= \int q_j(z_j) \mathbb{E}_{i \neq j}[\log p(\mathbf{x}, \mathbf{z})] dz_j + H(q_j) + \text{const}\end{aligned}$$

Derivation of the CAVI update

Isolate the terms involving q_j (everything else is a constant):

$$\begin{aligned}\mathcal{L}(q_j) &= \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z})] - \mathbb{E}_q[\log q(\mathbf{z})] \\ &= \int q_j(z_j) \left(\int \prod_{i \neq j} q_i(z_i) \log p(\mathbf{x}, \mathbf{z}) dz_{-j} \right) dz_j - \int q_j \log q_j dz_j + \text{const} \\ &= \int q_j(z_j) \mathbb{E}_{i \neq j}[\log p(\mathbf{x}, \mathbf{z})] dz_j + H(q_j) + \text{const}\end{aligned}$$

Define $\log \tilde{p}(z_j) \equiv \mathbb{E}_{i \neq j}[\log p(\mathbf{x}, \mathbf{z})] + \text{const}$. Then

$$\mathcal{L}(q_j) = -\text{KL}(q_j(z_j) \parallel \tilde{p}(z_j)) + \text{const}$$

Derivation of the CAVI update

Isolate the terms involving q_j (everything else is a constant):

$$\begin{aligned}\mathcal{L}(q_j) &= \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z})] - \mathbb{E}_q[\log q(\mathbf{z})] \\ &= \int q_j(z_j) \left(\int \prod_{i \neq j} q_i(z_i) \log p(\mathbf{x}, \mathbf{z}) dz_{-j} \right) dz_j - \int q_j \log q_j dz_j + \text{const} \\ &= \int q_j(z_j) \mathbb{E}_{i \neq j}[\log p(\mathbf{x}, \mathbf{z})] dz_j + H(q_j) + \text{const}\end{aligned}$$

Define $\log \tilde{p}(z_j) \equiv \mathbb{E}_{i \neq j}[\log p(\mathbf{x}, \mathbf{z})] + \text{const}$. Then

$$\mathcal{L}(q_j) = -\text{KL}(q_j(z_j) \parallel \tilde{p}(z_j)) + \text{const}$$

A KL is minimised (at zero) by matching the two arguments, so

$$q_j^*(z_j) \propto \exp \left(\mathbb{E}_{i \neq j} [\log p(\mathbf{x}, \mathbf{z})] \right) \blacksquare$$

```
def cavi(x, lam, n_sweeps=100, tol=1e-6):
    """lam[j] holds the variational parameters of factor q_j."""
    history = []
    for sweep in range(n_sweeps):
        for j in range(len(lam)):
            # q_j*(z_j) is proportional to exp( E_{i != j}[ log p(x, z) ] ).
            # For conjugate models this is a closed-form parameter update.
            lam[j] = update_factor(x, lam, j)

        history.append(elbo(x, lam))                # must increase every sweep
        if sweep > 0 and abs(history[-1] - history[-2]) < tol:
            break                                    # converged
    return lam, history
```

Example — Bayesian Gaussian mixture. With $q(c, \mu) = \prod_i q(c_i) \prod_k q(\mu_k)$ the two updates are responsibilities $q(c_i = k) \propto \exp\{\mathbb{E}[\mu_k]x_i - \frac{1}{2} \mathbb{E}[\mu_k^2]\}$ and a precision-weighted mean for $q(\mu_k)$. Both are one line of NumPy — but they had to be derived by hand for this specific model.

Mean-field VI: pros and cons

Pros

- **Fast:** typically orders of magnitude quicker than MCMC.
- **Deterministic:** monitor the ELBO and you know where you are.
- **Analytic updates** for conjugate models — no step sizes, no tuning.

Cons

- **The factorisation is wrong** in almost every real model.
- **Underestimates variance** — overconfident predictions.
- **Model-specific derivations:** change the likelihood and you re-derive everything by hand.

The third problem is the one that stopped VI from scaling. Next: how it was solved.

Where we are

The core problem: Bayesian inference

A brief tour of MCMC

Foundations of variational inference

Mean-field variational inference

Black box variational inference

Going deeper: hierarchical VAEs

Expressive approximations: normalizing flows

Score matching and diffusion models

Conclusion

The need for generality

Deriving CAVI updates by hand is a research bottleneck: every new likelihood means new algebra, and non-conjugate models have no closed form at all.

Question. Can we build a VI algorithm that only ever needs to *evaluate* $\log p(x, z)$ and differentiate it — treating the model as a black box?

The need for generality

Deriving CAVI updates by hand is a research bottleneck: every new likelihood means new algebra, and non-conjugate models have no closed form at all.

Question. Can we build a VI algorithm that only ever needs to *evaluate* $\log p(x, z)$ and differentiate it — treating the model as a black box?

Yes: stochastic gradient ascent on the ELBO.

$$\lambda_{t+1} = \lambda_t + \rho_t \widehat{\nabla_{\lambda} \mathcal{L}}(\lambda_t)$$

All the difficulty moves into one question: *how do we get an unbiased, low-variance estimate of that gradient?*

Why the ELBO gradient is awkward

$$\mathcal{L}(\lambda) = \mathbb{E}_{q(z; \lambda)} [\log p(x, z) - \log q(z; \lambda)]$$

$$\begin{aligned}\nabla_{\lambda} \mathcal{L} &= \nabla_{\lambda} \int q(z; \lambda) [\log p(x, z) - \log q(z; \lambda)] dz \\ &= \int \underbrace{\nabla_{\lambda} q(z; \lambda)}_{\text{not a density!}} [\dots] dz - \int q(z; \lambda) \nabla_{\lambda} \log q(z; \lambda) dz\end{aligned}$$

Why the ELBO gradient is awkward

$$\mathcal{L}(\lambda) = \mathbb{E}_{q(z; \lambda)} [\log p(x, z) - \log q(z; \lambda)]$$

$$\begin{aligned}\nabla_{\lambda} \mathcal{L} &= \nabla_{\lambda} \int q(z; \lambda) [\log p(x, z) - \log q(z; \lambda)] dz \\ &= \int \underbrace{\nabla_{\lambda} q(z; \lambda)}_{\text{not a density!}} [\dots] dz - \int q(z; \lambda) \nabla_{\lambda} \log q(z; \lambda) dz\end{aligned}$$

The obstacle

The parameters we differentiate sit in the *measure* we sample from. $\nabla_{\lambda} q$ is not a probability density, so we cannot just draw samples and average.

Two ways out: rewrite the gradient as an expectation (score function), or move the parameters out of the measure (reparameterisation).

Estimator 1: the score function (REINFORCE)

Use the log-derivative identity $\nabla_{\lambda} q = q \nabla_{\lambda} \log q$:

Score function estimator

$$\nabla_{\lambda} \mathcal{L} = \mathbb{E}_{q(z; \lambda)} \left[\nabla_{\lambda} \log q(z; \lambda) (\log p(x, z) - \log q(z; \lambda)) \right]$$

- Now it *is* an expectation under q : sample and average.
- **Extremely general**: works for discrete z , and $\log p$ need not be differentiable — we never differentiate the model.

Estimator 1: the score function (REINFORCE)

Use the log-derivative identity $\nabla_{\lambda} q = q \nabla_{\lambda} \log q$:

Score function estimator

$$\nabla_{\lambda} \mathcal{L} = \mathbb{E}_{q(z; \lambda)} \left[\nabla_{\lambda} \log q(z; \lambda) (\log p(x, z) - \log q(z; \lambda)) \right]$$

- Now it *is* an expectation under q : sample and average.
- **Extremely general**: works for discrete z , and $\log p$ need not be differentiable — we never differentiate the model.
- **But the variance is brutal**. The learning signal is a scalar reward multiplying a score; nothing tells the estimator *which direction* in z improved things.
- In practice it needs control variates (baselines), Rao-Blackwellisation and many samples.

Estimator 2: the reparameterisation trick

The trick

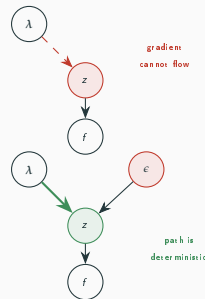
Do not sample z from q . Sample a parameter-free noise variable and *transform* it:

$$z = g(\epsilon, \lambda), \quad \epsilon \sim p(\epsilon)$$

where $p(\epsilon)$ does not depend on λ .

The Gaussian case

- $q(z; \mu, \sigma) = \mathcal{N}(z | \mu, \sigma^2)$
- Sample $\epsilon \sim \mathcal{N}(0, 1)$
- Set $z = \mu + \sigma\epsilon$
- Randomness now lives in ϵ ; μ and σ are just arithmetic on the path.



The reparameterisation gradient

The trick lets the gradient operator move *inside* the expectation:

$$\begin{aligned}\nabla_{\lambda} \mathcal{L} &= \nabla_{\lambda} \mathbb{E}_{q(z; \lambda)} [f(z)], & f(z) &= \log p(x, z) - \log q(z; \lambda) \\ &= \nabla_{\lambda} \mathbb{E}_{p(\epsilon)} [f(g(\epsilon, \lambda))] & & \text{(change of variables — the measure is now fixed)} \\ &= \mathbb{E}_{p(\epsilon)} [\nabla_{\lambda} f(g(\epsilon, \lambda))] & & \text{(swap } \nabla \text{ and } \int)\end{aligned}$$

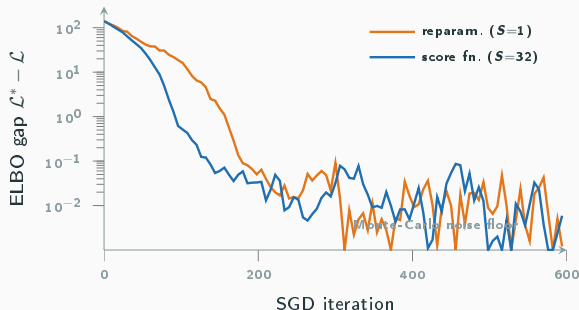
The estimator

$$\widehat{\nabla_{\lambda} \mathcal{L}} = \frac{1}{S} \sum_{s=1}^S \nabla_{\lambda} f(g(\epsilon^{(s)}, \lambda)), \quad \epsilon^{(s)} \sim p(\epsilon)$$

It uses $\nabla_z \log p(x, z)$ — *directional* information about where the density increases. That is why its variance is so much smaller. Autodiff computes it for free; often $S = 1$ suffices.

The two estimators, measured

A 1-D conjugate model ($z \sim \mathcal{N}(0, 1)$, $x_i | z \sim \mathcal{N}(z, 1)$, $N = 10$), where the exact posterior and ELBO are known.



Note the cost: every point on the blue curve used 32 model evaluations, every point on the orange curve 1.

Gradient standard deviation at the optimum (40k samples):

	$\partial/\partial\mu$	$\partial/\partial\log\sigma$
Reparam.	3.3	1.4
Score function	16.8	7.2
ratio	$5.1\times$	$5.1\times$

$5\times$ the standard deviation is $26\times$ the variance — so roughly $26\times$ the samples for the same gradient quality.

With $S = 1$ the score-function run diverges entirely (σ blows up past 10^4) at the step size where reparameterisation converges in ~ 50 iterations.

BBVI with the reparameterisation trick, in PyTorch

```
import torch
from torch.distributions import Normal

mu          = torch.zeros(D, requires_grad=True)      # variational parameters
log_sigma   = torch.zeros(D, requires_grad=True)      # log for positivity
opt = torch.optim.Adam([mu, log_sigma], lr=1e-2)

for step in range(2000):
    sigma = log_sigma.exp()
    eps    = torch.randn(S, D)                        # base noise: no parameters
    z       = mu + sigma * eps                          # <-- reparameterisation

    log_q = Normal(mu, sigma).log_prob(z).sum(-1)      # gradient flows through z AND q
    elbo   = (log_joint(x, z) - log_q).mean()          # one-sample MC estimate

    opt.zero_grad()
    (-elbo).backward()                                # autodiff does the chain rule
    opt.step()
```

Nothing here mentions the model. Swap `log_joint` and the same 12 lines perform inference in a completely different model — that is the “black box”.

The score-function estimator, in PyTorch

```
z      = Normal(mu, sigma).sample()           # .sample() -- NO path gradient
log_q  = Normal(mu, sigma).log_prob(z).sum(-1)

f = (log_joint(x, z) - log_q).detach()        # learning signal, treated as constant
b = f.mean()                                  # control variate (baseline)

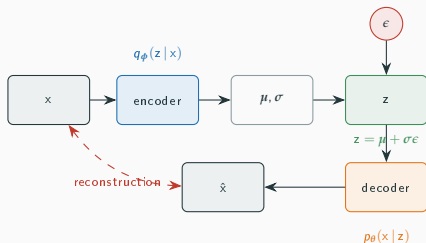
surrogate = (log_q * (f - b)).mean()          # d/dlambda of this == REINFORCE grad
(-surrogate).backward()
```

- `.sample()` instead of `.rsample()`: the sample is a constant, the gradient enters only through $\log q$.
- `.detach()` on the signal is essential — otherwise autodiff silently computes a different (biased) gradient.
- Use this only when you must: **discrete latent variables**, non-differentiable models, or reinforcement learning.

Estimator	Requires	Variance	Handles discrete z
Score function (REINFORCE)	differentiable $\log q$	high	yes
Reparameterisation	differentiable g and $\log p$	low	no

- Prefer reparameterisation whenever the latent space is continuous.
- Combine with **minibatching** of the data and VI scales to millions of observations (stochastic VI, Hoffman et al., 2013).
- This is what made the VAE possible — and with it the modern era of deep generative models.

Everything so far, in one architecture: the VAE



$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi} [\log p_\theta(x|z)] \\ - \text{KL}(q_\phi(z|x) \parallel p(z))$$

- **Amortisation:** one network outputs $\lambda = (\mu, \sigma)$ for every data point, instead of optimising λ_i per point.
- The KL term is analytic for Gaussian q and prior.
- Trained end-to-end by exactly the loop two slides ago.

Where we are

The core problem: Bayesian inference

A brief tour of MCMC

Foundations of variational inference

Mean-field variational inference

Black box variational inference

Going deeper: hierarchical VAEs

Expressive approximations: normalizing flows

Score matching and diffusion models

Conclusion

Why one stochastic layer is not enough

The VAE has exactly one latent variable, with a fixed $\mathcal{N}(0, I)$ prior. Two things break because of it.

The prior is too simple

The model can only generate what $p(z) = \mathcal{N}(0, I)$ can reach after one decoder pass. Real data lives on a curved, clustered manifold; a single Gaussian ball pushed through one network struggles to cover it.

The posterior is too simple

$q(z | x)$ is a diagonal Gaussian — mean-field all over again. The ELBO gap $\text{KL}(q \parallel p(z | x))$ stays large, so the bound on $\log p(x)$ stays loose.

Why one stochastic layer is not enough

The VAE has exactly one latent variable, with a fixed $\mathcal{N}(0, I)$ prior. Two things break because of it.

The prior is too simple

The model can only generate what $p(z) = \mathcal{N}(0, I)$ can reach after one decoder pass. Real data lives on a curved, clustered manifold; a single Gaussian ball pushed through one network struggles to cover it.

The posterior is too simple

$q(z | x)$ is a diagonal Gaussian — mean-field all over again. The ELBO gap $\text{KL}(q \parallel p(z | x))$ stays large, so the bound on $\log p(x)$ stays loose.

The idea

Stack latent variables: $z_L \rightarrow z_{L-1} \rightarrow \dots \rightarrow z_1 \rightarrow x$. Each layer is a simple conditional Gaussian, but their *composition* is not.

Same trick as normalizing flows — depth instead of width — but the layers here are stochastic.

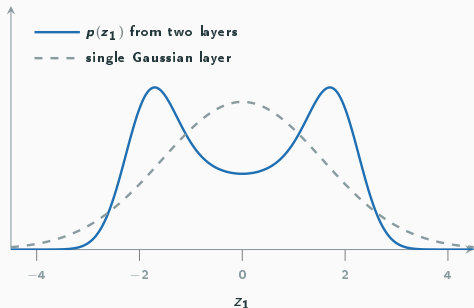
The effective prior gets richer with depth

A two-layer hierarchy with *Gaussian* pieces:

$$p(z_2) = \mathcal{N}(0, 1)$$

$$p(z_1 | z_2) = \mathcal{N}(\mu(z_2), \sigma^2)$$

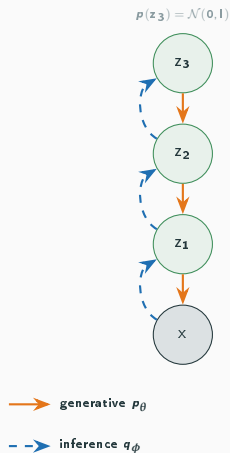
$$p(z_1) = \int p(z_1 | z_2) p(z_2) dz_2$$



- The marginal is **bimodal** — no single Gaussian layer can produce it.
- This is an infinite mixture of Gaussians, indexed by z_2 .
- z_1 never sees a fixed prior: its prior is *generated* by the layer above.

(Computed by quadrature with $\mu(z_2) = 2 \tanh(1.6z_2)$, $\sigma = 0.45$.)

The hierarchical model



Generative model (top-down)

$$p_\theta(x, z_{1:L}) = p(z_L) \prod_{l=1}^{L-1} p_\theta(z_l | z_{l+1}) p_\theta(x | z_1)$$

Inference model (bottom-up)

$$q_\phi(z_{1:L} | x) = \prod_{l=1}^L q_\phi(z_l | z_{l-1}), \quad z_0 \equiv x$$

- Only $p(z_L)$ is a fixed $\mathcal{N}(0, 1)$; every other prior is *learned* and conditional.
- Every conditional is still a diagonal Gaussian — so every piece is reparameterisable and the whole thing trains with the same loop.

The hierarchical ELBO

Nothing new is needed — write down the ELBO and group the terms by layer:

$$\begin{aligned}\mathcal{L} &= \mathbb{E}_q [\log p_\theta(x, z_{1:L}) - \log q_\phi(z_{1:L} | x)] \\ &= \mathbb{E}_q [\log p_\theta(x | z_1)] + \sum_{l=1}^{L-1} \mathbb{E}_q [\log p_\theta(z_l | z_{l+1}) - \log q_\phi(z_l | z_{l-1})] \\ &\quad + \mathbb{E}_q [\log p(z_L) - \log q_\phi(z_L | z_{L-1})]\end{aligned}$$

Nothing new is needed — write down the ELBO and group the terms by layer:

$$\begin{aligned}\mathcal{L} &= \mathbb{E}_q [\log p_\theta(x, z_{1:L}) - \log q_\phi(z_{1:L} | x)] \\ &= \mathbb{E}_q [\log p_\theta(x | z_1)] + \sum_{l=1}^{L-1} \mathbb{E}_q [\log p_\theta(z_l | z_{l+1}) - \log q_\phi(z_l | z_{l-1})] \\ &\quad + \mathbb{E}_q [\log p(z_L) - \log q_\phi(z_L | z_{L-1})]\end{aligned}$$

The hierarchical ELBO, layer by layer

Layer-wise form

$$\begin{aligned}\mathcal{L} = & \underbrace{\mathbb{E}_q [\log p_\theta(x | z_1)]}_{\text{reconstruction}} - \underbrace{\text{KL} (q_\phi(z_L | z_{L-1}) \parallel p(z_L))}_{\text{top layer, against } \mathcal{N}(0, I)} \\ & - \sum_{l=1}^{L-1} \underbrace{\mathbb{E}_q [\text{KL} (q_\phi(z_l | z_{l-1}) \parallel p_\theta(z_l | z_{l+1}))]}_{\text{one KL per layer, against a } \textit{learned} \text{ prior}}\end{aligned}$$

The hierarchical ELBO, layer by layer

Layer-wise form

$$\begin{aligned}\mathcal{L} = & \underbrace{\mathbb{E}_q [\log p_\theta(x | z_1)]}_{\text{reconstruction}} - \underbrace{\text{KL} (q_\phi(z_L | z_{L-1}) \parallel p(z_L))}_{\text{top layer, against } \mathcal{N}(0, I)} \\ & - \sum_{l=1}^{L-1} \underbrace{\mathbb{E}_q [\text{KL} (q_\phi(z_l | z_{l-1}) \parallel p_\theta(z_l | z_{l+1}))]}_{\text{one KL per layer, against a learned prior}}\end{aligned}$$

- One reconstruction term and L KL terms — each between two diagonal Gaussians, so each is still analytic and still one line of code.
- Only the *top* KL pulls towards $\mathcal{N}(0, I)$. Every other layer is compared against a prior the model itself learns, which is exactly where the extra expressiveness comes from.
- Reading the KL terms tells you how many nats each layer actually stores. A layer with $\text{KL}_l \approx 0$ is dead weight — more on that shortly.

Wiring the inference network: bottom-up is a bad match

Naive: bottom-up q

$$q(z_I | z_{I-1})$$

- Information flows $x \rightarrow z_1 \rightarrow \dots \rightarrow z_L$.
- But the generative model flows the *other* way, so the true posterior of z_I depends on z_{I+1} — which this q never looks at.
- Upper layers receive learning signal only through a long chain. In practice they die (next slide).

Wiring the inference network: bottom-up is a bad match

Naive: bottom-up q

$$q(z_l | z_{l-1})$$

- Information flows $x \rightarrow z_1 \rightarrow \dots \rightarrow z_L$.
- But the generative model flows the *other* way, so the true posterior of z_l depends on z_{l+1} — which this q never looks at.
- Upper layers receive learning signal only through a long chain. In practice they die (next slide).

Better: top-down q (Ladder VAE, NVAE, VDVAE)

$$q(z_l | z_{>l}, x)$$

- A bottom-up pass extracts features from x once; the *sampling* then follows the same top-down path as the generative model, sharing its parameters.
- Parameterise q as a **correction to the prior**:

$$q = \mathcal{N}(\mu_p + \Delta\mu, \sigma_p \cdot \Delta\sigma)$$

- At initialisation $\Delta\mu \approx 0$, $\Delta\sigma \approx 1$, so $q \approx p$ and every KL starts near zero — the optimisation begins from a *feasible* point instead of fighting its way there.

The failure mode: posterior collapse

What happens

For some layer l the optimiser finds

$$q_{\phi}(z_l | \cdot) = p_{\theta}(z_l | z_{l+1}) \implies \text{KL}_l = 0$$

The layer carries **zero information** about x and the decoder ignores it. Depth is bought and never used.

Why it is tempting. The ELBO pays a KL price for every nat of information a layer stores. If a powerful decoder can explain x without z_l , setting $\text{KL}_l = 0$ is simply the cheaper local optimum — especially early in training, when the decoder is still bad and the latents are still noise.

The standard remedies

- **KL warm-up:** weight the KL terms by $\beta_t : 0 \rightarrow 1$ over the first few epochs. Learn to reconstruct first, pay for information later.
- **Free bits:** replace KL_l by $\max(\lambda, \text{KL}_l)$ — the first λ nats per layer are free, so there is no gradient pushing them to zero.
- **Residual parameterisation** of q (previous slide).
- **Skip connections** from every z_l to the output, so each layer has a short path to the reconstruction term.

A hierarchical VAE, in PyTorch

```
def hvae_elbo(x, layers, decoder, beta=1.0, free_bits=0.1):
    """layers: top-down list; layers[0] is the topmost latent z_L."""
    ctx = bottom_up(x)                # features from x, one per layer
    h = top_state()                   # learned constant: start of the top-down pass
    kls = []

    for l, layer in enumerate(layers): # z_L, z_{L-1}, ..., z_1
        p_mu, p_sd = layer.prior(h)    # p(z_l | z_{l+1}) -- learned, not N(0, I)
        d_mu, d_sd = layer.posterior(h, ctx[l])
        q_mu, q_sd = p_mu + d_mu, p_sd * d_sd # residual: q starts out equal to p

        eps = torch.randn_like(q_mu)
        z = q_mu + q_sd * eps           # reparameterisation, same as before
        kls.append(kl_normal(q_mu, q_sd, p_mu, p_sd).sum(-1))

    h = layer.merge(h, z)              # carry z_l down the top-down path

    recon = decoder(h).log_prob(x).sum(-1)
    kl = sum(torch.clamp(k, min=free_bits) for k in kls) # free bits per layer
    return (recon - beta * kl).mean()    # beta: 0 -> 1 warm-up
```

The generative path (`layer.prior`) and the inference path (`layer.posterior`) share the same state `h` — that is the whole top-down trick.

Where this leads

Very deep VAEs

- **NVAE** (2020) and **VDVAE** (2021) stack *dozens* of stochastic layers.
- VDVAE with 78 layers beats autoregressive models on likelihood, and it is still just the ELBO from four slides ago.
- The engineering is all about stability: residual q , spectral normalisation, gradient skipping, careful initialisation.

The punchline

A **diffusion model** is a **hierarchical VAE** with three things given away for free:

- the encoder $q(x_t | x_{t-1})$ is *fixed* — add Gaussian noise, no parameters to learn;
- every latent has the *same shape* as the data;
- one network is *shared* across all T levels, conditioned on t .

Its training objective is exactly the layer-wise ELBO on this slide.

Keep this picture: the diffusion section is about to arrive at the same equation from the opposite direction.

Where we are

The core problem: Bayesian inference

A brief tour of MCMC

Foundations of variational inference

Mean-field variational inference

Black box variational inference

Going deeper: hierarchical VAEs

Expressive approximations: normalizing flows

Score matching and diffusion models

Conclusion

Normalizing flows: the core idea

Mean-field and Gaussian q are simply too rigid. Can we build an expressive q that still has a tractable density?

From simple to complex by transformation

Push a simple distribution through a chain of *invertible, differentiable* maps.

1. Start from $z_0 \sim q_0$, e.g. $\mathcal{N}(0, I)$.

Normalizing flows: the core idea

Mean-field and Gaussian q are simply too rigid. Can we build an expressive q that still has a tractable density?

From simple to complex by transformation

Push a simple distribution through a chain of *invertible, differentiable* maps.

1. Start from $z_0 \sim q_0$, e.g. $\mathcal{N}(0, I)$.
2. Apply K bijections: $z_k = f_k(z_{k-1})$.

Normalizing flows: the core idea

Mean-field and Gaussian q are simply too rigid. Can we build an expressive q that still has a tractable density?

From simple to complex by transformation

Push a simple distribution through a chain of *invertible, differentiable* maps.

1. Start from $z_0 \sim q_0$, e.g. $\mathcal{N}(0, I)$.
2. Apply K bijections: $z_k = f_k(z_{k-1})$.
3. Use the law of $z_K = f_K \circ \dots \circ f_1(z_0)$ as the variational distribution q_K .

Normalizing flows: the core idea

Mean-field and Gaussian q are simply too rigid. Can we build an expressive q that still has a tractable density?

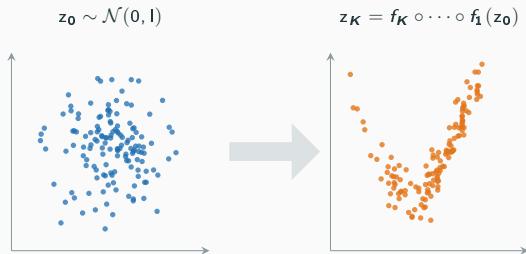
From simple to complex by transformation

Push a simple distribution through a chain of *invertible, differentiable* maps.

1. Start from $z_0 \sim q_0$, e.g. $\mathcal{N}(0, I)$.
2. Apply K bijections: $z_k = f_k(z_{k-1})$.
3. Use the law of $z_K = f_K \circ \dots \circ f_1(z_0)$ as the variational distribution q_K .

Invertibility buys us the exact density; differentiability buys us gradients.

A flow, in pictures



Same 140 points, pushed through four planar/affine layers. The density is now curved, skewed and multimodal — yet we can still write down $\log q_K$ exactly.

The change of variables formula

For one bijection $z_1 = f_1(z_0)$, probability mass is conserved but volume is not:

$$q_1(z_1) = q_0(z_0) \left| \det \frac{\partial f_1}{\partial z_0} \right|^{-1}$$

The change of variables formula

For one bijection $z_1 = f_1(z_0)$, probability mass is conserved but volume is not:

$$q_1(z_1) = q_0(z_0) \left| \det \frac{\partial f_1}{\partial z_0} \right|^{-1}$$

Taking logs:

$$\log q_1(z_1) = \log q_0(z_0) - \log \left| \det \frac{\partial f_1}{\partial z_0} \right|$$

The change of variables formula

For one bijection $z_1 = f_1(z_0)$, probability mass is conserved but volume is not:

$$q_1(z_1) = q_0(z_0) \left| \det \frac{\partial f_1}{\partial z_0} \right|^{-1}$$

Taking logs:

$$\log q_1(z_1) = \log q_0(z_0) - \log \left| \det \frac{\partial f_1}{\partial z_0} \right|$$

Log-density of a K -step flow

$$\log q_K(z_K) = \log q_0(z_0) - \sum_{k=1}^K \log \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right|$$

The whole game is designing f_k that are expressive *and* whose Jacobian determinant costs $O(D)$ instead of $O(D^3)$.

Using flows for variational inference

1. Let λ be the parameters of all transformations $\{f_k\}$.

Using flows for variational inference

1. Let λ be the parameters of all transformations $\{f_k\}$.
2. The ELBO becomes

$$\mathcal{L}(\lambda) = \mathbb{E}_{q_0(z_0)} \left[\log p(x, z_K) - \log q_0(z_0) + \sum_{k=1}^K \log \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right| \right]$$

(the expectation is over the *base* distribution — the reparameterisation trick, for free)

Using flows for variational inference

1. Let λ be the parameters of all transformations $\{f_k\}$.
2. The ELBO becomes

$$\mathcal{L}(\lambda) = \mathbb{E}_{q_0(z_0)} \left[\log p(x, z_K) - \log q_0(z_0) + \sum_{k=1}^K \log \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right| \right]$$

(the expectation is over the *base* distribution — the reparameterisation trick, for free)

3. Estimate it by Monte Carlo:
 - 3.1 Sample $z_0 \sim q_0$.
 - 3.2 Push it forward, accumulating $\log |\det J_k|$ along the way.
 - 3.3 Evaluate $\log p(x, z_K)$.

Using flows for variational inference

1. Let λ be the parameters of all transformations $\{f_k\}$.
2. The ELBO becomes

$$\mathcal{L}(\lambda) = \mathbb{E}_{q_0(z_0)} \left[\log p(x, z_K) - \log q_0(z_0) + \sum_{k=1}^K \log \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right| \right]$$

(the expectation is over the *base* distribution — the reparameterisation trick, for free)

3. Estimate it by Monte Carlo:
 - 3.1 Sample $z_0 \sim q_0$.
 - 3.2 Push it forward, accumulating $\log |\det J_k|$ along the way.
 - 3.3 Evaluate $\log p(x, z_K)$.
4. Backpropagate and take a gradient step. Nothing else changes.

Example flow architectures

- **Planar flows** — stretch/contract along one direction:

$$f(z) = z + u h(w^\top z + b), \quad |\det J| = |1 + u^\top \psi(z)|$$

Rank-one update, so the matrix determinant lemma gives $O(D)$.

Example flow architectures

- **Planar flows** — stretch/contract along one direction:

$$f(z) = z + u h(w^\top z + b), \quad |\det J| = |1 + u^\top \psi(z)|$$

Rank-one update, so the matrix determinant lemma gives $O(D)$.

- **Radial flows** — push mass towards or away from a point:

$$f(z) = z + \beta h(\alpha, r)(z - z_{\text{ref}})$$

Example flow architectures

- **Planar flows** — stretch/contract along one direction:

$$f(z) = z + u h(w^\top z + b), \quad |\det J| = |1 + u^\top \psi(z)|$$

Rank-one update, so the matrix determinant lemma gives $O(D)$.

- **Radial flows** — push mass towards or away from a point:

$$f(z) = z + \beta h(\alpha, r)(z - z_{\text{ref}})$$

- **Autoregressive flows** (MAF, IAF) — condition each dimension on the previous ones:

$$z_i = \mu_i + \sigma_i \epsilon_i, \quad (\mu_i, \sigma_i) = \text{NN}(z_{1:i-1})$$

The Jacobian is triangular, so $\det = \prod_i \sigma_i$. Very expressive; sampling and density evaluation have opposite cost profiles.

Example flow architectures

- **Planar flows** — stretch/contract along one direction:

$$f(z) = z + u h(w^\top z + b), \quad |\det J| = |1 + u^\top \psi(z)|$$

Rank-one update, so the matrix determinant lemma gives $O(D)$.

- **Radial flows** — push mass towards or away from a point:

$$f(z) = z + \beta h(\alpha, r)(z - z_{\text{ref}})$$

- **Autoregressive flows** (MAF, IAF) — condition each dimension on the previous ones:

$$z_i = \mu_i + \sigma_i \epsilon_i, \quad (\mu_i, \sigma_i) = \text{NN}(z_{1:i-1})$$

The Jacobian is triangular, so $\det = \prod_i \sigma_i$. Very expressive; sampling and density evaluation have opposite cost profiles.

- **Coupling layers** (RealNVP, Glow) — split the vector in two, transform one half conditioned on the other. Fast in *both* directions, which is why they dominate in practice.

A planar flow layer, in PyTorch

```
class PlanarFlow(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.u = nn.Parameter(torch.randn(d) * 0.01)
        self.w = nn.Parameter(torch.randn(d) * 0.01)
        self.b = nn.Parameter(torch.zeros(1))

    def u_hat(self):
        # constrain w.u >= -1 so f is invertible
        wu = self.w @ self.u
        m = -1 + torch.nn.functional.softplus(wu)
        return self.u + (m - wu) * self.w / (self.w @ self.w + 1e-8)

    def forward(self, z):
        # z: (batch, d)
        u = self.u_hat()
        a = torch.tanh(z @ self.w + self.b)
        # (batch,)
        f = z + u * a.unsqueeze(-1)
        # the transformation
        psi = (1 - a**2).unsqueeze(-1) * self.w
        # h'(.) * w
        log_det = torch.log(torch.abs(1 + psi @ u) + 1e-8)
        return f, log_det
        # accumulate log_det over layers
```

Where we are

The core problem: Bayesian inference

A brief tour of MCMC

Foundations of variational inference

Mean-field variational inference

Black box variational inference

Going deeper: hierarchical VAEs

Expressive approximations: normalizing flows

Score matching and diffusion models

Conclusion

Beyond explicit densities

VAEs and flows both model an explicit, *normalised* density. That normalisation is expensive: flows must stay invertible with cheap Jacobians, VAEs settle for a lower bound.

Beyond explicit densities

VAEs and flows both model an explicit, *normalised* density. That normalisation is expensive: flows must stay invertible with cheap Jacobians, VAEs settle for a lower bound.

A different paradigm

What if we never model the density at all — only its **gradient**?

Beyond explicit densities

VAEs and flows both model an explicit, *normalised* density. That normalisation is expensive: flows must stay invertible with cheap Jacobians, VAEs settle for a lower bound.

A different paradigm

What if we never model the density at all — only its **gradient**?

The score function

$$s(x) = \nabla_x \log p(x)$$

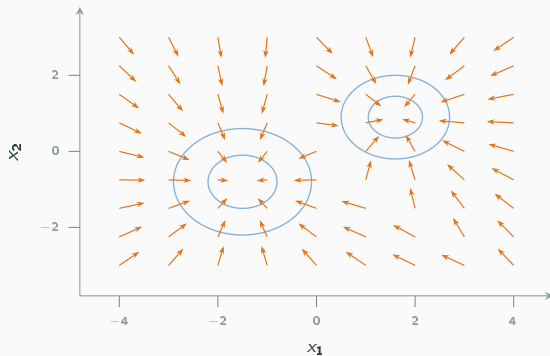
The gradient is taken with respect to *the data* x , not the parameters.

Why this is the right object: if $p(x) = \tilde{p}(x) / Z$, then

$$\nabla_x \log p(x) = \nabla_x \log \tilde{p}(x) - \underbrace{\nabla_x \log Z}_{=0}$$

The intractable constant differentiates away.

The score is a vector field



- At every point, $\nabla_x \log p(x)$ points **uphill** in log-density.
- Knowing this field is enough to *navigate* to the data manifold — we never need to know how much probability is actually there.
- Arrows vanish at the modes: that is where the field is a fixed point.

The plan

Train a network $s_\theta(x)$ to output this field, then follow it (plus noise) to generate samples.

The goal: score matching

Train a network $s_{\theta}(x)$ to match the true score:

Explicit score matching

$$J(\theta) = \frac{1}{2} \mathbb{E}_{p_{\text{data}}(x)} \left[\|s_{\theta}(x) - \nabla_x \log p_{\text{data}}(x)\|_2^2 \right]$$

The goal: score matching

Train a network $s_{\theta}(x)$ to match the true score:

Explicit score matching

$$J(\theta) = \frac{1}{2} \mathbb{E}_{p_{\text{data}}(x)} \left[\|s_{\theta}(x) - \nabla_x \log p_{\text{data}}(x)\|_2^2 \right]$$

The problem

We do not know $\nabla_x \log p_{\text{data}}(x)$. We only have samples. The target of our regression is unavailable.

The goal: score matching

Train a network $s_\theta(x)$ to match the true score:

Explicit score matching

$$J(\theta) = \frac{1}{2} \mathbb{E}_{p_{\text{data}}(x)} \left[\|s_\theta(x) - \nabla_x \log p_{\text{data}}(x)\|_2^2 \right]$$

The problem

We do not know $\nabla_x \log p_{\text{data}}(x)$. We only have samples. The target of our regression is unavailable.

- Hyvärinen (2005) removed the unknown by integration by parts, at the cost of a Hessian trace $\text{tr}(\nabla_x s_\theta) — O(D)$ backward passes. Hopeless for images.
- **Vincent (2011)** found a way that costs nothing: *denoising* score matching.

Denoising score matching

Match the score of **noise-perturbed** data instead of clean data.

1. Perturb: $q_{\sigma}(\tilde{x} | x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I)$.

Denoising score matching

Match the score of **noise-perturbed** data instead of clean data.

1. Perturb: $q_{\sigma}(\tilde{x} | x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I)$.
2. This defines $q_{\sigma}(\tilde{x}) = \int p_{\text{data}}(x) q_{\sigma}(\tilde{x} | x) dx$ — a smoothed version of the data distribution.

Denoising score matching

Match the score of **noise-perturbed** data instead of clean data.

1. Perturb: $q_{\sigma}(\tilde{x} | x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I)$.
2. This defines $q_{\sigma}(\tilde{x}) = \int p_{\text{data}}(x) q_{\sigma}(\tilde{x} | x) dx$ — a smoothed version of the data distribution.
3. Target its score $\nabla_{\tilde{x}} \log q_{\sigma}(\tilde{x})$.

Denoising score matching

Match the score of **noise-perturbed** data instead of clean data.

1. Perturb: $q_\sigma(\tilde{x} | x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I)$.
2. This defines $q_\sigma(\tilde{x}) = \int p_{\text{data}}(x) q_\sigma(\tilde{x} | x) dx$ — a smoothed version of the data distribution.
3. Target its score $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x})$.

The key identity (Vincent, 2011)

Up to a constant independent of θ ,

$$\mathbb{E}_{q_\sigma(\tilde{x})} \|s_\theta(\tilde{x}) - \nabla \log q_\sigma(\tilde{x})\|^2 = \mathbb{E}_{q_\sigma(\tilde{x}, x)} \|s_\theta(\tilde{x}) - \nabla \log q_\sigma(\tilde{x} | x)\|^2$$

The intractable marginal score can be replaced by the *conditional* score.

Denoising score matching

Match the score of **noise-perturbed** data instead of clean data.

1. Perturb: $q_\sigma(\tilde{x} | x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I)$.
2. This defines $q_\sigma(\tilde{x}) = \int p_{\text{data}}(x) q_\sigma(\tilde{x} | x) dx$ — a smoothed version of the data distribution.
3. Target its score $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x})$.

The key identity (Vincent, 2011)

Up to a constant independent of θ ,

$$\mathbb{E}_{q_\sigma(\tilde{x})} \|s_\theta(\tilde{x}) - \nabla \log q_\sigma(\tilde{x})\|^2 = \mathbb{E}_{q_\sigma(\tilde{x}, x)} \|s_\theta(\tilde{x}) - \nabla \log q_\sigma(\tilde{x} | x)\|^2$$

The intractable marginal score can be replaced by the *conditional* score.

And for Gaussian noise the conditional score is trivial:

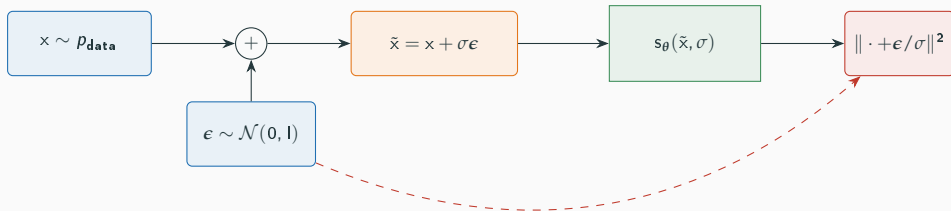
$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} | x) = -\frac{\tilde{x} - x}{\sigma^2} = -\frac{\epsilon}{\sigma}$$

DSM is just denoising

The loss

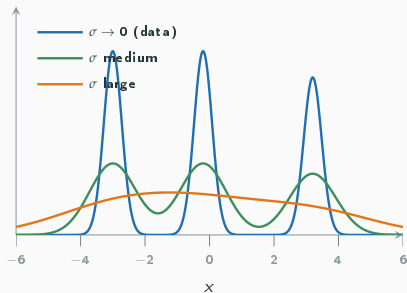
$$J_{\text{DSM}}(\theta) = \frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}, \epsilon \sim \mathcal{N}(0, I)} \left[\|s_{\theta}(x + \sigma\epsilon) + \epsilon/\sigma\|_2^2 \right]$$

Predicting the score is the same as predicting the noise that was added.



A state-of-the-art generative model is trained as a denoising autoencoder.

One noise level is not enough



- **Small σ :** sharp and accurate near the data — but in the empty regions between modes there are no training samples, so the learned score is garbage exactly where a sampler starts.
- **Large σ :** the modes merge into one smooth blob. The score is reliable everywhere, but it points to a blurred version of the truth.

Solution: condition on σ

One network $s_\theta(x, \sigma)$, trained on a geometric ladder $\sigma_1 > \sigma_2 > \dots > \sigma_L$.

Noise conditional score networks (NCSN)

Training objective

$$J(\theta) = \frac{1}{L} \sum_{i=1}^L \lambda(\sigma_i) \mathbb{E} \left[\|s_{\theta}(\tilde{x}_i, \sigma_i) + \epsilon / \sigma_i\|_2^2 \right], \quad \lambda(\sigma_i) = \sigma_i^2$$

- The weighting $\lambda(\sigma) = \sigma^2$ makes every noise level contribute on the same scale — without it the small- σ terms (whose targets are huge, $\propto 1/\sigma$) dominate the loss.
- In practice: for each item in a minibatch, draw a random level σ_i . One network, one loss, one training run.
- Conditioning is usually done by a sinusoidal embedding of σ (or of the diffusion timestep t) fed into every residual block of a U-Net.

Denoising score matching, in PyTorch

```
def dsm_loss(model, x, sigmas):  
    """x: (B, C, H, W); sigmas: (L,) decreasing geometric ladder."""  
    i = torch.randint(len(sigmas), (x.shape[0],), device=x.device)  
    sigma = sigmas[i].view(-1, 1, 1, 1) # one level per example  
  
    eps = torch.randn_like(x)  
    x_tilde = x + sigma * eps # perturb  
    target = -eps / sigma # grad log q(x_tilde | x)  
  
    score = model(x_tilde, sigma) # the network  
    # weight by sigma^2 so all noise levels contribute equally  
    return 0.5 * (sigma**2 * (score - target)**2).flatten(1).sum(1).mean()
```

Equivalently, let the network predict ϵ directly and the loss becomes plain `mse(eps_hat, eps)` — which is exactly the DDPM training objective.

Sampling: Langevin dynamics

We have a score field. How do we turn it into samples?

Langevin dynamics

$$x_{t+1} = x_t + \frac{\alpha}{2} \nabla_x \log p(x_t) + \sqrt{\alpha} z_t, \quad z_t \sim \mathcal{N}(0, I)$$

As $\alpha \rightarrow 0$ and $t \rightarrow \infty$, x_t is distributed as p .

- **Drift term:** climb the log-density — move towards the modes.
- **Noise term:** without it the iteration is just gradient ascent and collapses onto a single mode. The noise is what makes it *sampling* rather than optimisation.

Sampling: Langevin dynamics

We have a score field. How do we turn it into samples?

Langevin dynamics

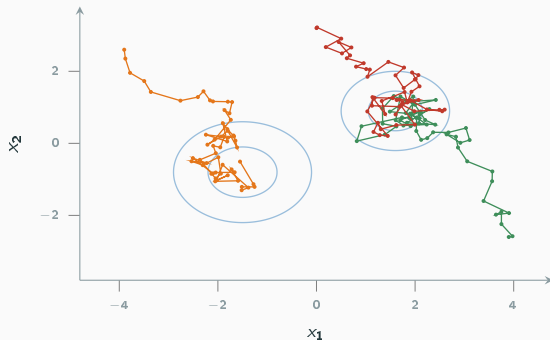
$$x_{t+1} = x_t + \frac{\alpha}{2} \nabla_x \log p(x_t) + \sqrt{\alpha} z_t, \quad z_t \sim \mathcal{N}(0, I)$$

As $\alpha \rightarrow 0$ and $t \rightarrow \infty$, x_t is distributed as p .

- **Drift term:** climb the log-density — move towards the modes.
- **Noise term:** without it the iteration is just gradient ascent and collapses onto a single mode. The noise is what makes it *sampling* rather than optimisation.
- It only needs the *score*, so we can substitute our network:

$$x_{t+1} = x_t + \frac{\alpha}{2} s_{\theta}(x_t) + \sqrt{\alpha} z_t$$

Langevin dynamics, in pictures



Three chains started far from the data, each following s_θ with injected noise.

- They are pulled into the modes within a few dozen steps...
- ...and then *stay there and jitter*, which is precisely what a sample from the distribution should do.

The catch

With a single noise level, a chain that starts in an empty region follows a score the network never learned, and the relative weight of the two modes comes out wrong.

Algorithm

1. **Initialise** from pure noise: $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \sigma_1^2 \mathbf{I})$.

Algorithm

1. **Initialise** from pure noise: $x \sim \mathcal{N}(0, \sigma_1^2 \mathbf{I})$.
2. **For each level** $i = 1, \dots, L$ (largest noise first):
 - step size $\alpha_i = \epsilon \cdot \sigma_i^2 / \sigma_L^2$
 - run T Langevin steps

$$x \leftarrow x + \frac{\alpha_i}{2} s_{\theta}(x, \sigma_i) + \sqrt{\alpha_i} z$$

Algorithm

1. **Initialise** from pure noise: $x \sim \mathcal{N}(0, \sigma_1^2 \mathbf{I})$.
2. **For each level** $i = 1, \dots, L$ (largest noise first):
 - step size $\alpha_i = \epsilon \cdot \sigma_i^2 / \sigma_L^2$
 - run T Langevin steps

$$x \leftarrow x + \frac{\alpha_i}{2} s_\theta(x, \sigma_i) + \sqrt{\alpha_i} z$$

3. **Return** x after the last (smallest-noise) level.

The chain starts on a heavily blurred version of the data distribution, where the score is well-defined everywhere and the modes are connected. As σ shrinks, the landscape sharpens and the sample is refined — coarse structure first, fine detail last.

Annealed Langevin sampling, in PyTorch

```
@torch.no_grad()
def annealed_langevin(model, shape, sigmas, T=100, eps=2e-5):
    """sigmas: decreasing, e.g. geospace(50, 0.01, 10)."""
    x = torch.rand(shape)  # start from pure noise

    for sigma in sigmas:  # coarse -> fine
        alpha = eps * (sigma / sigmas[-1])**2  # step size scales with sigma^2
        for t in range(T):
            z = torch.randn_like(x)
            x = x + 0.5 * alpha * model(x, sigma) + alpha.sqrt() * z

    return x
```

$L \times T$ network evaluations per sample — typically 1000 or more. This is the price of the sample quality, and the reason distillation and fast ODE solvers (DDIM, DPM-Solver, consistency models) are such an active area.

Connection to diffusion models

Score-based models (NCSN, SDE)

- Continuous time: a forward SDE gradually turns data into noise.
- Train: learn $\nabla \log p_t(x_t)$ at every time t .
- Sample: solve the reverse-time SDE (or its probability-flow ODE).

Diffusion models (DDPM)

- Discrete time: a Markov chain adds Gaussian noise over T steps.
- Train: maximise an **ELBO** on the reverse chain — which reduces to $\|\epsilon - \epsilon_\theta\|^2$.
- Sample: simulate the learned reverse chain from noise.

Connection to diffusion models

Score-based models (NCSN, SDE)

- Continuous time: a forward SDE gradually turns data into noise.
- Train: learn $\nabla \log p_t(x_t)$ at every time t .
- Sample: solve the reverse-time SDE (or its probability-flow ODE).

Diffusion models (DDPM)

- Discrete time: a Markov chain adds Gaussian noise over T steps.
- Train: maximise an **ELBO** on the reverse chain — which reduces to $\|\epsilon - \epsilon_\theta\|^2$.
- Sample: simulate the learned reverse chain from noise.

The unification

The DDPM noise predictor and the score model are the same thing up to scaling:

$$s_\theta(x_t, t) = -\frac{\epsilon_\theta(x_t, t)}{\sigma_t}$$

And note where we started: the DDPM objective is an ELBO. **Diffusion models are variational inference** over a fixed, hand-designed inference chain $q(x_{1:T} | x_0)$.

Score-based models: pros and cons

Pros

- **State-of-the-art samples** for images, audio, video, molecules.
- **No architectural constraints:** no invertibility, no tractable Jacobian — any U-Net or transformer works.
- **No partition function** anywhere in training.
- **Stable training:** it is a regression loss, not a minimax game.

Cons

- **Slow sampling:** hundreds to thousands of network evaluations (improving fast).
- **No cheap likelihood:** requires solving the probability-flow ODE.
- **No encoder:** no latent representation comes for free, unlike a VAE.

Generative model families: a comparison

Property	VAE	Normalizing flow	Score / diffusion	GAN
Primary goal	approximate inference	density estimation	sample generation	sample generation
Sample quality	good, often blurry	good, can be rigid	state of the art	very good
Sampling speed	fast (one shot)	fast (one shot)	slow (iterative)	fast (one shot)
Latent encoder	yes, built in	yes (invertible)	no	no
Likelihood	lower bound (ELBO)	exact	ODE, expensive	none
Training	stable	stable	stable	adversarial, fragile
Key idea	maximise the ELBO	change of variables	learn the score	fool a discriminator

Three of these four are trained by maximising the likelihood or a bound on it — and two of them are literally variational inference.

Where we are

The core problem: Bayesian inference

A brief tour of MCMC

Foundations of variational inference

Mean-field variational inference

Black box variational inference

Going deeper: hierarchical VAEs

Expressive approximations: normalizing flows

Score matching and diffusion models

Conclusion

Summary: VI vs. MCMC

Variational inference	Markov chain Monte Carlo
Inference as <i>optimisation</i> .	Inference as <i>sampling</i> .
Fast; scales to large data with minibatches.	Slow; every step touches all the data.
Approximate — accuracy is capped by the family Q .	Asymptotically exact.
Deterministic; monitor the ELBO to check convergence.	Stochastic; convergence can only be falsified.
Underestimates posterior variance (reverse KL).	Captures variance, correlations and multimodality.
Gives a lower bound on the evidence.	Gives no evidence estimate directly.

Rule of thumb: MCMC when the model is small and the answer must be right; VI when the model is large and the answer must exist.

The journey of approximate inference

- **The core problem:** the evidence integral is intractable, so the posterior is known only up to a constant.

The journey of approximate inference

- **The core problem:** the evidence integral is intractable, so the posterior is known only up to a constant.
- **Two escapes:** sampling (**MCMC**) or optimisation (**VI**).

The journey of approximate inference

- **The core problem:** the evidence integral is intractable, so the posterior is known only up to a constant.
- **Two escapes:** sampling (**MCMC**) or optimisation (**VI**).
- **Classic VI: mean-field** + CAVI — fast, but restrictive and re-derived by hand for every model.

The journey of approximate inference

- **The core problem:** the evidence integral is intractable, so the posterior is known only up to a constant.
- **Two escapes:** sampling (**MCMC**) or optimisation (**VI**).
- **Classic VI: mean-field** + CAVI — fast, but restrictive and re-derived by hand for every model.
- **Modern VI: BBVI** + the **reparameterisation trick** — general, autodiff-friendly, scalable. This is what made VAEs possible.

The journey of approximate inference

- **The core problem:** the evidence integral is intractable, so the posterior is known only up to a constant.
- **Two escapes:** sampling (**MCMC**) or optimisation (**VI**).
- **Classic VI: mean-field** + CAVI — fast, but restrictive and re-derived by hand for every model.
- **Modern VI: BBVI** + the **reparameterisation trick** — general, autodiff-friendly, scalable. This is what made VAEs possible.
- **Expressive VI: normalizing flows** — flexible families with exact densities, via the change of variables formula.

The journey of approximate inference

- **The core problem:** the evidence integral is intractable, so the posterior is known only up to a constant.
- **Two escapes:** sampling (**MCMC**) or optimisation (**VI**).
- **Classic VI: mean-field** + CAVI — fast, but restrictive and re-derived by hand for every model.
- **Modern VI: BBVI** + the **reparameterisation trick** — general, autodiff-friendly, scalable. This is what made VAEs possible.
- **Expressive VI: normalizing flows** — flexible families with exact densities, via the change of variables formula.
- **Implicit models: score matching** and **diffusion** — drop the density, learn its gradient, and generate by following the field.

The journey of approximate inference

- **The core problem:** the evidence integral is intractable, so the posterior is known only up to a constant.
- **Two escapes:** sampling (**MCMC**) or optimisation (**VI**).
- **Classic VI: mean-field** + CAVI — fast, but restrictive and re-derived by hand for every model.
- **Modern VI: BBVI** + the **reparameterisation trick** — general, autodiff-friendly, scalable. This is what made VAEs possible.
- **Expressive VI: normalizing flows** — flexible families with exact densities, via the change of variables formula.
- **Implicit models: score matching** and **diffusion** — drop the density, learn its gradient, and generate by following the field.

Every step traded something: exactness for speed, flexibility for tractability, likelihood for sample quality. **There is no free lunch — only a well-stocked menu.**

Variational inference

- Blei, Kucukelbir & McAuliffe (2017). *Variational Inference: A Review for Statisticians*. JASA. — the best single entry point.
- Jordan et al. (1999). *An Introduction to Variational Methods for Graphical Models*.
- Hoffman, Blei, Wang & Paisley (2013). *Stochastic Variational Inference*. JMLR.
- Ranganath, Gerrish & Blei (2014). *Black Box Variational Inference*. AISTATS.

Reparameterisation, flows, and beyond

- Kingma & Welling (2014). *Auto-Encoding Variational Bayes*. ICLR.
- Rezende & Mohamed (2015). *Variational Inference with Normalizing Flows*. ICML.
- Papamakarios et al. (2021). *Normalizing Flows for Probabilistic Modeling and Inference*. JMLR.

Hierarchical VAEs

- Sønderby et al. (2016). *Ladder Variational Autoencoders*. NeurIPS. — top-down inference.
- Kingma et al. (2016). *Improved VI with Inverse Autoregressive Flow*. NeurIPS. — free bits.
- Vahdat & Kautz (2020). *NVAE: A Deep Hierarchical Variational Autoencoder*. NeurIPS.
- Child (2021). *Very Deep VAEs Generalize Autoregressive Models...* ICLR.
- Luo (2022). *Understanding Diffusion Models: A Unified Perspective*. — derives diffusion as an HVAE.

Score matching and diffusion

- Hyvärinen (2005). *Estimation of Non-Normalized Statistical Models by Score Matching*. JMLR.
- Vincent (2011). *A Connection Between Score Matching and Denoising Autoencoders*. Neural Computation.
- Song & Ermon (2019). *Generative Modeling by Estimating Gradients of the Data Distribution*. NeurIPS.
- Ho, Jain & Abbeel (2020). *Denoising Diffusion Probabilistic Models*. NeurIPS.
- Song et al. (2021). *Score-Based Generative Modeling through SDEs*. ICLR.

Thank you

Questions?

`andrijaster@gmail.com`